# Computer Architecture Lab

## (EL-2012)

## LABORATORY REPORT

## Fall 2026




# (Lab#07)

## Implementation of Arithmetic Logic Unit (ALU)

Student Name: *Syed Sikandar Abbas* Roll No: *24i-6553* Section: *CE-B*

Date performed: *6/10*/, 2026

**MARKS AWARDED: _____ / 10**

**Signature and date of Instructor: _____**
**Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad**

## Task 1:

**Description:**

Using the given design, write a Verilog code at all level of abstraction to make a Module for 1 bit ALU having inputs (1 bit) as A and B as input data, carry_in, select line (2 bit input) to select between different operations i.e. AND, OR and ADD/SUB. The module will have an Output for Result of operations and another output for carry_out. Selection between ADD/SUB will be provided by another input b_inv. Show simulation results for every instruction executed. Make sure to design a circuit diagram using logisim and a truth table to verify the working of ALU.

**Code:**

# Gate Level:

```
module myAND(
    output O,
    input A, B, C, D
);

wire t0, t1;
and(t0, A, B);
and(t1, C, D);
and(O, t0, t1);

endmodule

module Multiplexer_GLevel(
    output O,
    input S0, S1, I0, I1, I2, I3
);

wire SN0, SN1;
not(SN0, S0);
not(SN1, S1);

wire t0, t1, t2, t3;

myAND A1(t0, I0, SN0, SN1, 1);
myAND A2(t1, I1, SN0, S1, 1);
myAND A3(t2, I2, S0, SN1, 1);
myAND A4(t3, I3, S0, S1, 1);

wire to0, to1;
or(to0, t0, t1);
or(to1, to0, t2);
or(O, t3, to1); //or'ed 3 inputs using 3 2-input modules

endmodule

module HalfAdder_GLevel(Sum, Carry, A, B);
```

```
input A, B;
output Sum, Carry;

xor sum(Sum, A, B);
and carry(Carry, A, B);

endmodule

module FullAdder_GLevel(Sum, Carry, A, B, Cin);

input A, B, Cin;
output Sum, Carry;
wire s1, c1, c2;

HalfAdder_GLevel ha1(s1, c1, A, B);
HalfAdder_GLevel ha2(Sum, c2, s1, Cin);
or carry(Carry, c1, c2);

endmodule

// -----------MAIN MODULE-----------------
module ALU_GLevel(A, B, carry_in, result, carry_out, S0, S1, b_inv);

input A, B, S0, S1, b_inv, carry_in;
output result, carry_out;

wire b_final;
wire fa_sum;
wire fa_carry;
wire And;
wire Or;

xor (b_final, B, b_inv); //b final will be used in the circuit

FullAdder_GLevel fa1(fa_sum, fa_carry, A, b_final, carry_in);

and a(And, A, B);
or o(Or, A, B);

/*
S0 S1
0  0 - AND
0  1 - OR
1  0 - ADD/SUB
1  1 - ADD/SUB
*/

Multiplexer_GLevel mux(result, S0, S1, And, Or, fa_sum, fa_sum);
assign carry_out = fa_carry;

endmodule


module _1bitALU_tb;
```

```
reg A, B, Cin;
reg b_inv;
reg S0, S1;
wire O, Cout;

ALU_GLevel myALU(A, B, Cin, O, Cout, S0, S1, b_inv);

initial
begin
         A = 0; B = 0; Cin = 0; b_inv = 0; S0 = 0; S1 = 0; //0 AND 0 = 0
    #10; A = 1; B = 1; S0 = 0; S1 = 0; b_inv = 0; //1 AND 1 = 1
    #10; A = 0; B = 1; S0 = 0; S1 = 1; b_inv = 0; //0 OR 1 = 1
    #10; A = 1; B = 1; S0 = 0; S1 = 1; b_inv = 0; //1 OR 1 = 1
    #10; A = 1; B = 1; Cin = 0; S0 = 1; S1 = 0; b_inv = 0; //1 + 1 = 10
    #10; A = 1; B = 1; Cin = 1; S0 = 1; S1 = 0; b_inv = 0; //1 + 1 + 1 = 11
    #10; A = 1; B = 0; Cin = 1; S0 = 1; S1 = 0; b_inv = 1; //1 - 0 + 1 = 1 + 1 + 1 =
11
    #10; A = 0; B = 1; Cin = 1; S0 = 1; S1 = 1; b_inv = 1; //0 - 1 + 1 = 0 + 1 + 1 =
10
end

endmodule
```

# DataFlow Level:

```
module myAND(
    output O,
    input A, B, C, D
);

assign O = A & B & C & D;

endmodule

module Multiplexer_Dataflow(
    output O,
    input S0, S1, I0, I1, I2, I3
);

assign O = (~S1 & ~S0 & I0) | (~S1 & S0 & I1) | (S1 & ~S0 & I2) | (S1 & S0 &
I3);

endmodule

module HalfAdder_Dataflow(Sum, Carry, A, B);

input A, B;
output Sum, Carry;

assign Sum = A ^ B;
assign Carry = A & B;
```

```
endmodule

module FullAdder_Dataflow(Sum, Carry, A, B, Cin);

input A, B, Cin;
output Sum, Carry;

assign Sum = A ^ B ^ Cin;
assign Carry = (A & B) | (B & Cin) | (A & Cin);

endmodule

// -----------MAIN MODULE------------------
module ALU_Dataflow(A, B, carry_in, result, carry_out, S0, S1, b_inv);

input A, B, S0, S1, b_inv, carry_in;
output result, carry_out;

wire b_final;
wire fa_sum;
wire fa_carry;
wire And;
wire Or;

assign b_final = B ^ b_inv;
assign {fa_carry, fa_sum} = A + b_final + carry_in;
assign And = A & B;
assign Or = A | B;
assign result = ({S1, S0} == 2'b00) ? And : ({S1, S0} == 2'b01) ? Or : fa_sum;
assign carry_out = fa_carry;

endmodule

module _1bitALU_tb;

reg A, B, Cin;
reg b_inv;
reg S0, S1;
wire O, Cout;

ALU_Dataflow myALU(A, B, Cin, O, Cout, S0, S1, b_inv);

initial
begin
         A = 0; B = 0; Cin = 0; b_inv = 0; S0 = 0; S1 = 0;
    #10; A = 1; B = 1; S0 = 0; S1 = 0; b_inv = 0;
    #10; A = 0; B = 1; S0 = 0; S1 = 1; b_inv = 0;
    #10; A = 1; B = 1; S0 = 0; S1 = 1; b_inv = 0;
    #10; A = 1; B = 1; Cin = 0; S0 = 1; S1 = 0; b_inv = 0;
```

```
    #10; A = 1; B = 1; Cin = 1; S0 = 1; S1 = 0; b_inv = 0;
    #10; A = 1; B = 0; Cin = 1; S0 = 1; S1 = 0; b_inv = 1;
    #10; A = 0; B = 1; Cin = 1; S0 = 1; S1 = 1; b_inv = 1;
end

endmodule
```

# Behavioural Level:

```
module myAND(
    output reg O,
    input A, B, C, D
);

always @(*) begin
    O = A & B & C & D;
end

endmodule

module Multiplexer_Behavioral(
    output reg O,
    input S0, S1, I0, I1, I2, I3
);

always @(*) begin
    case ({S1, S0})
        2'b00: O = I0;
        2'b01: O = I1;
        2'b10: O = I2;
        2'b11: O = I3;
        default: O = 1'b0;
    endcase
end

endmodule

module HalfAdder_Behavioral(Sum, Carry, A, B);

input A, B;
output reg Sum, Carry;

always @(*) begin
    Sum = A ^ B;
    Carry = A & B;
end

endmodule
```

```
module FullAdder_Behavioral(Sum, Carry, A, B, Cin);

input A, B, Cin;
output reg Sum, Carry;

always @(*) begin
    {Carry, Sum} = A + B + Cin;
end

endmodule

// -----------MAIN MODULE------------------
module ALU_Behavioral(A, B, carry_in, result, carry_out, S0, S1, b_inv);

input A, B, S0, S1, b_inv, carry_in;
output reg result, carry_out;

reg b_final;
reg fa_sum;
reg fa_carry;
reg And;
reg Or;

always @(*) begin
    b_final = B ^ b_inv;
    {fa_carry, fa_sum} = A + b_final + carry_in;
    And = A & B;
    Or = A | B;

    case ({S1, S0})
        2'b00: result = And;
        2'b01: result = Or;
        2'b10: result = fa_sum;
        2'b11: result = fa_sum;
        default: result = 1'b0;
    endcase

    carry_out = fa_carry;
end

endmodule

module _1bitALU_tb;

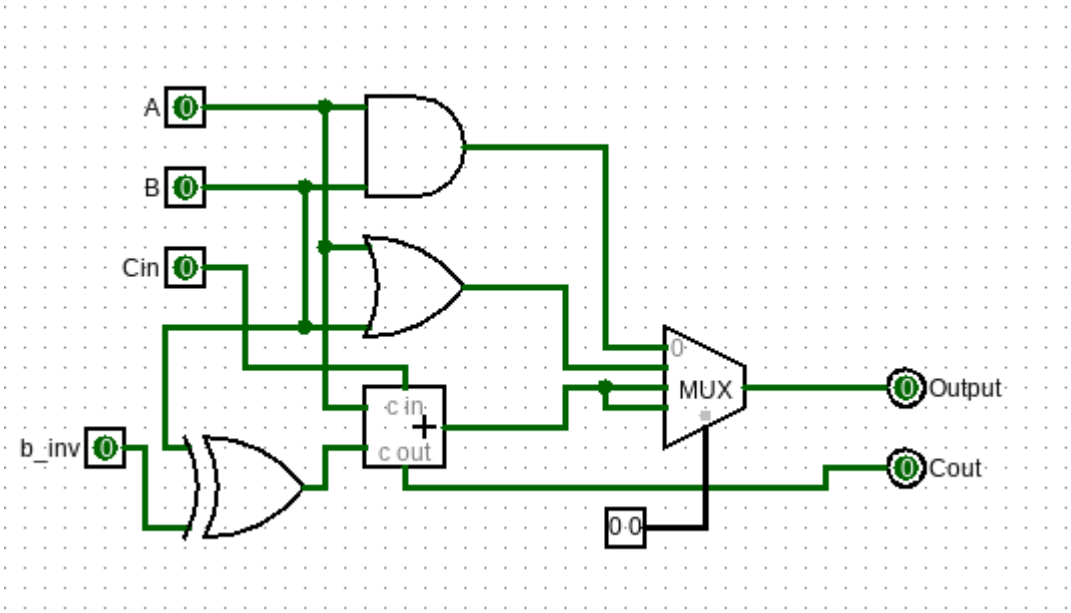
reg A, B, Cin;
reg b_inv;
reg S0, S1;
wire O, Cout;

ALU_Behavioral myALU(A, B, Cin, O, Cout, S0, S1, b_inv);
```

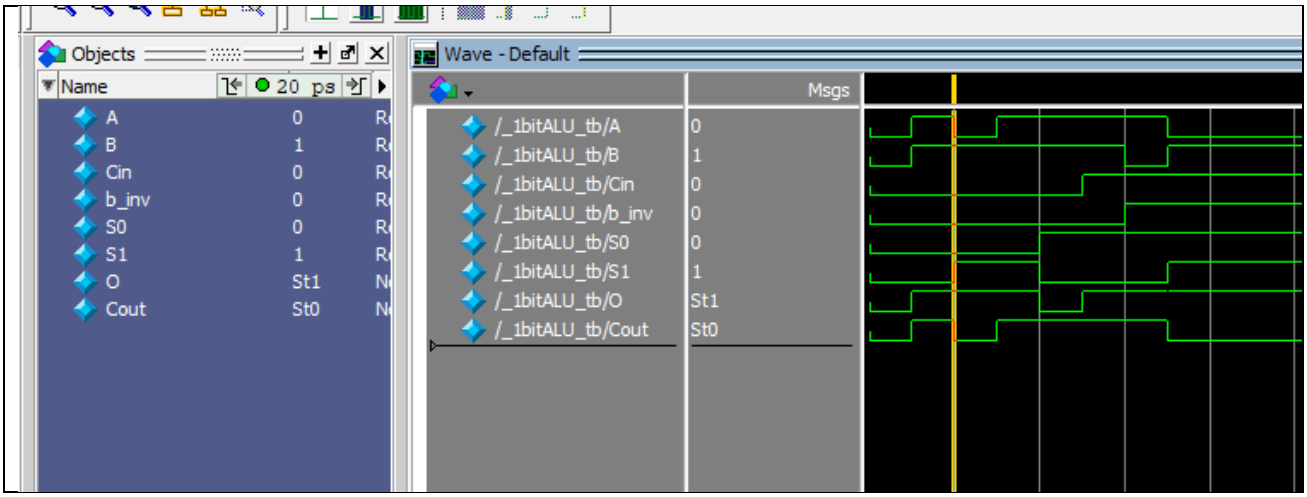
```
initial
begin
         A = 0; B = 0; Cin = 0; b_inv = 0; S0 = 0; S1 = 0;
    #10; A = 1; B = 1; S0 = 0; S1 = 0; b_inv = 0;
    #10; A = 0; B = 1; S0 = 0; S1 = 1; b_inv = 0;
    #10; A = 1; B = 1; S0 = 0; S1 = 1; b_inv = 0;
    #10; A = 1; B = 1; Cin = 0; S0 = 1; S1 = 0; b_inv = 0;
    #10; A = 1; B = 1; Cin = 1; S0 = 1; S1 = 0; b_inv = 0;
    #10; A = 1; B = 0; Cin = 1; S0 = 1; S1 = 0; b_inv = 1;
    #10; A = 0; B = 1; Cin = 1; S0 = 1; S1 = 1; b_inv = 1;
end

endmodule
```

# Logisim:




**Output:**


Objects
Name
20 ps
A 0
B 1
Cin 0
b_inv 0
S0 0
S1 1
O St1
Cout St0
Wave - Default
Msgs
/_1bitALU_tb/A 0
/_1bitALU_tb/B 1
/_1bitALU_tb/Cin 0
/_1bitALU_tb/b_inv 0
/_1bitALU_tb/S0 0
/_1bitALU_tb/S1 1
/_1bitALU_tb/O St1
/_1bitALU_tb/Cout St0

## Task 2:

**Description:**

Write Verilog code to build a 4bit ALU out of 4 1bit ALUs. The ALU must be capable of performing ADD, SUB operations and logical operation that are AND and OR operations and a SLT operation. Status flags (CF, SF, ZF and OF) must also be set accordingly Test the code using test bench and simulation.

**Code:**

```
module myAND(
    output O,
    input A, B, C, D
);
    wire t0, t1;
    and(t0, A, B);
    and(t1, C, D);
    and(O, t0, t1);
endmodule

module Multiplexer_GLevel(
    output O,
    input S0, S1, I0, I1, I2, I3
);
    wire SN0, SN1;
    not(SN0, S0);
    not(SN1, S1);

    wire t0, t1, t2, t3;

    myAND A1(t0, I0, SN0, SN1, 1);
    myAND A2(t1, I1, SN0, S1, 1);
    myAND A3(t2, I2, S0, SN1, 1);
    myAND A4(t3, I3, S0, S1, 1);

    wire to0, to1;
    or(to0, t0, t1);
    or(to1, to0, t2);
    or(O, t3, to1);
endmodule

module HalfAdder_GLevel(Sum, Carry, A, B);
    input A, B;
    output Sum, Carry;

    xor sum(Sum, A, B);
    and carry(Carry, A, B);
endmodule

module FullAdder_GLevel(Sum, Carry, A, B, Cin);
    input A, B, Cin;
    output Sum, Carry;
    wire s1, c1, c2;

    HalfAdder_GLevel ha1(s1, c1, A, B);
    HalfAdder_GLevel ha2(Sum, c2, s1, Cin);
    or carry(Carry, c1, c2);
endmodule
```

```
// ----------- 1-BIT ALU MODULE ------------------
module ALU_1bit_GLevel(A, B, carry_in, result, carry_out, S0, S1, b_inv, less);
    input A, B, S0, S1, b_inv, carry_in, less;
    output result, carry_out;

    wire b_final;
    wire fa_sum;
    wire fa_carry;
    wire And;
    wire Or;

    xor (b_final, B, b_inv);
    FullAdder_GLevel fa1(fa_sum, fa_carry, A, b_final, carry_in);

    and a(And, A, B);
    or o(Or, A, B);

    // Multiplexer selects:
    // 00: AND, 01: OR, 10: ADD/SUB, 11: SLT
    Multiplexer_GLevel mux(result, S0, S1, And, Or, fa_sum, less);
    assign carry_out = fa_carry;
endmodule

// ----------- 4-BIT ALU MODULE ------------------
module ALU_4bit_GLevel(
    input [3:0] A,
    input [3:0] B,
    input S0, S1, b_inv, carry_in,
    output [3:0] result,
    output carry_out,
    output cf, sf, zf, of
);
    wire [4:0] c;
    wire [3:0] fa_results;
    wire slt_val;

    assign c[0] = carry_in | b_inv;
    assign slt_val = sf ^ of;

    ALU_1bit_GLevel alu0(A[0], B[0], c[0], result[0], c[1], S0, S1, b_inv, slt_val);
    ALU_1bit_GLevel alu1(A[1], B[1], c[1], result[1], c[2], S0, S1, b_inv, 1'b0);
    ALU_1bit_GLevel alu2(A[2], B[2], c[2], result[2], c[3], S0, S1, b_inv, 1'b0);
    ALU_1bit_GLevel alu3(A[3], B[3], c[3], result[3], c[4], S0, S1, b_inv, 1'b0);

    assign carry_out = c[4];

    assign cf = c[4] & (S0); // Carry Flag active during arithmetic
    assign sf = result[3]; // Sign Flag (MSB of result)
    assign zf = ~(|result); // Zero Flag (1 if all bits are 0)
    assign of = c[4] ^ c[3]; // Overflow Flag (C4 ^ C3)

endmodule

// ----------- TESTBENCH ------------------
module ALU_4bit_tb;

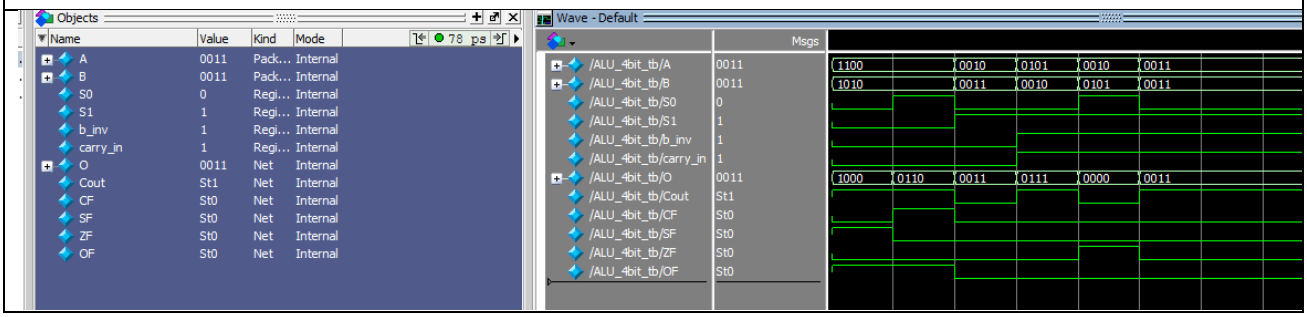
    reg [3:0] A, B;
    reg S0, S1, b_inv, carry_in;
    wire [3:0] O;
    wire Cout, CF, SF, ZF, OF;
```

```
    ALU_4bit_GLevel myALU4(A, B, S0, S1, b_inv, carry_in, O, Cout, CF, SF, ZF, OF);

    initial begin
             A = 4'b1100; B = 4'b1010; S0 = 0; S1 = 0; b_inv = 0; carry_in = 0; //1100 & 1010 =
1000
        #10; A = 4'b1100; B = 4'b1010; S0 = 1; S1 = 0; b_inv = 0; carry_in = 0; //1100 | 1010 =
1110
        #10; A = 4'b0010; B = 4'b0011; S0 = 0; S1 = 1; b_inv = 0; carry_in = 0; //0010 + 0011 =
0101
        #10; A = 4'b0101; B = 4'b0010; S0 = 0; S1 = 1; b_inv = 1; carry_in = 1; //0101 - 0010 =
0011
        #10; A = 4'b0010; B = 4'b0101; S0 = 1; S1 = 1; b_inv = 1; carry_in = 1; //0010 < 0101 ->
true, so result = 0001
        #10; A = 4'b0011; B = 4'b0011; S0 = 0; S1 = 1; b_inv = 1; carry_in = 1; //0011 - 0011 =
0000 -> ZF=1
    end

endmodule
```

**Output:**



## Task 3:

**Description:**

Write a Verilog code for a parameterized ALU, where the parameter N specifies the bit-width of the input and output signals. The operation of the ALU is determined by a separate control signal. The ALU should support the following operations: a. Addition b. Subtraction c. Basic logical operations (AND, OR, NOT, XOR) The inputs and outputs of the ALU must be N-bit wide, making the design scalable and reusable for different bit sizes.

**Code:**

```
module Parameterized_ALU #(parameter N = 4) (
    input [N-1:0] A,
    input [N-1:0] B,
    input [2:0] op,
    output reg [N-1:0] O,
    output reg Cout,
    output reg ZF,
    output reg SF,
    output reg OF
);

    always @(*) begin
        Cout = 0;
        OF = 0;
        case(op)
            3'b000: {Cout, O} = A + B;
            3'b001: {Cout, O} = A - B;
            3'b010: O = A & B;
            3'b011: O = A | B;
            3'b100: O = ~A;
            3'b101: O = A ^ B;
            default: O = 0;
        endcase
        ZF = (O == 0);
        SF = O[N-1];
        if (op == 3'b000)
            OF = (A[N-1] == B[N-1]) && (O[N-1] != A[N-1]);
        else if (op == 3'b001)
            OF = (A[N-1] != B[N-1]) && (O[N-1] != A[N-1]);
    end

endmodule

module ALU_4bit_tb;

    reg [3:0] A, B;
    reg [2:0] op;
```

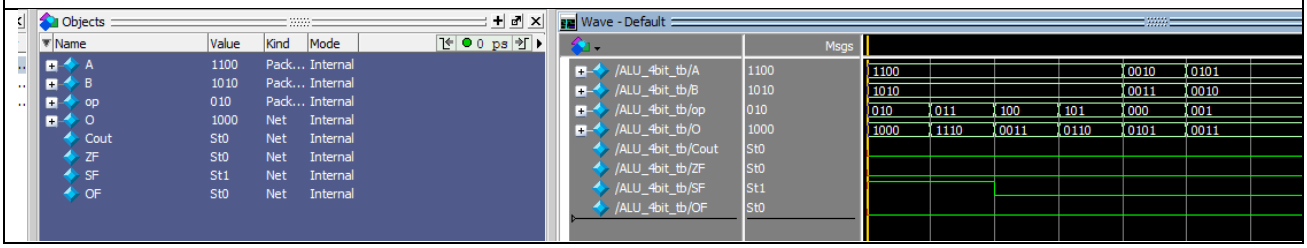
```
    wire [3:0] O;
    wire Cout, ZF, SF, OF;

    Parameterized_ALU #(4) myALU4(A, B, op, O, Cout, ZF, SF, OF);

    initial begin
             A = 4'b1100; B = 4'b1010; op = 3'b010; // 1100 & 1010 = 1000
        #10; A = 4'b1100; B = 4'b1010; op = 3'b011; // 1100 | 1010 = 1110
        #10; A = 4'b1100; B = 4'b1010; op = 3'b100; // ~1100 = 0011
        #10; A = 4'b1100; B = 4'b1010; op = 3'b101; // 1100 ^ 1010 = 0110
        #10; A = 4'b0010; B = 4'b0011; op = 3'b000; // 0010 + 0011 = 0101
        #10; A = 4'b0101; B = 4'b0010; op = 3'b001; // 0101 - 0010 = 0011
    end

endmodule
```

**Output:**



## Task 4:

**Description:**

Design and model a fully functional 32-bit ALU using Verilog. The ALU must perform a specified set of arithmetic, logical, and shifting operations (given in table 1) based on a set of control signals, and correctly manage status flags (given in table 2).

**Code:**

```
module alu_32bit (
    input [31:0] a,
    input [31:0] b,
    input [3:0] alu_op,
    output reg [31:0] result,
    output zero
);

    always @(*) begin
        case (alu_op)
            4'b0000: result = a + b;                                    // ADD
            4'b0001: result = a - b;                                    // SUB
            4'b0010: result = a & b;                                    // AND
            4'b0011: result = a | b;                                    // OR
            4'b0100: result = a ^ b;                                    // XOR
            4'b0101: result = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0; // SLT
            4'b0110: result = (a < b) ? 32'd1 : 32'd0;                   // SLTU
            4'b0111: result = a << b[4:0];                               // SLL
            4'b1000: result = a >> b[4:0];                               // SRL
            4'b1001: result = $signed(a) >>> b[4:0];                     // SRA
            default: result = 32'd0;
        endcase
    end

    assign zero = (result == 32'd0);

endmodule

module ALU_32bit_tb;

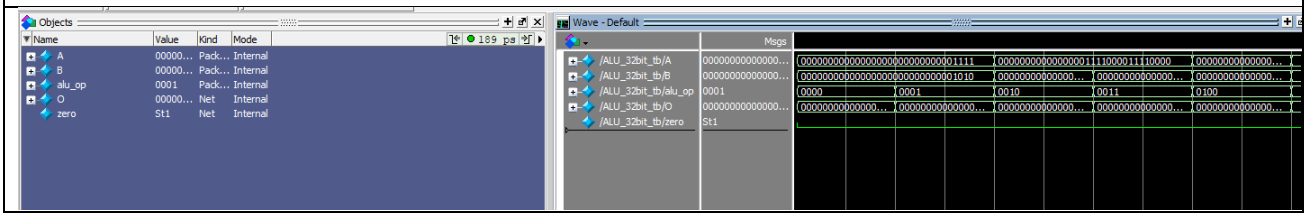
    reg [31:0] A, B;
    reg [3:0] alu_op;
    wire [31:0] O;
    wire zero;

    alu_32bit myALU32(A, B, alu_op, O, zero);

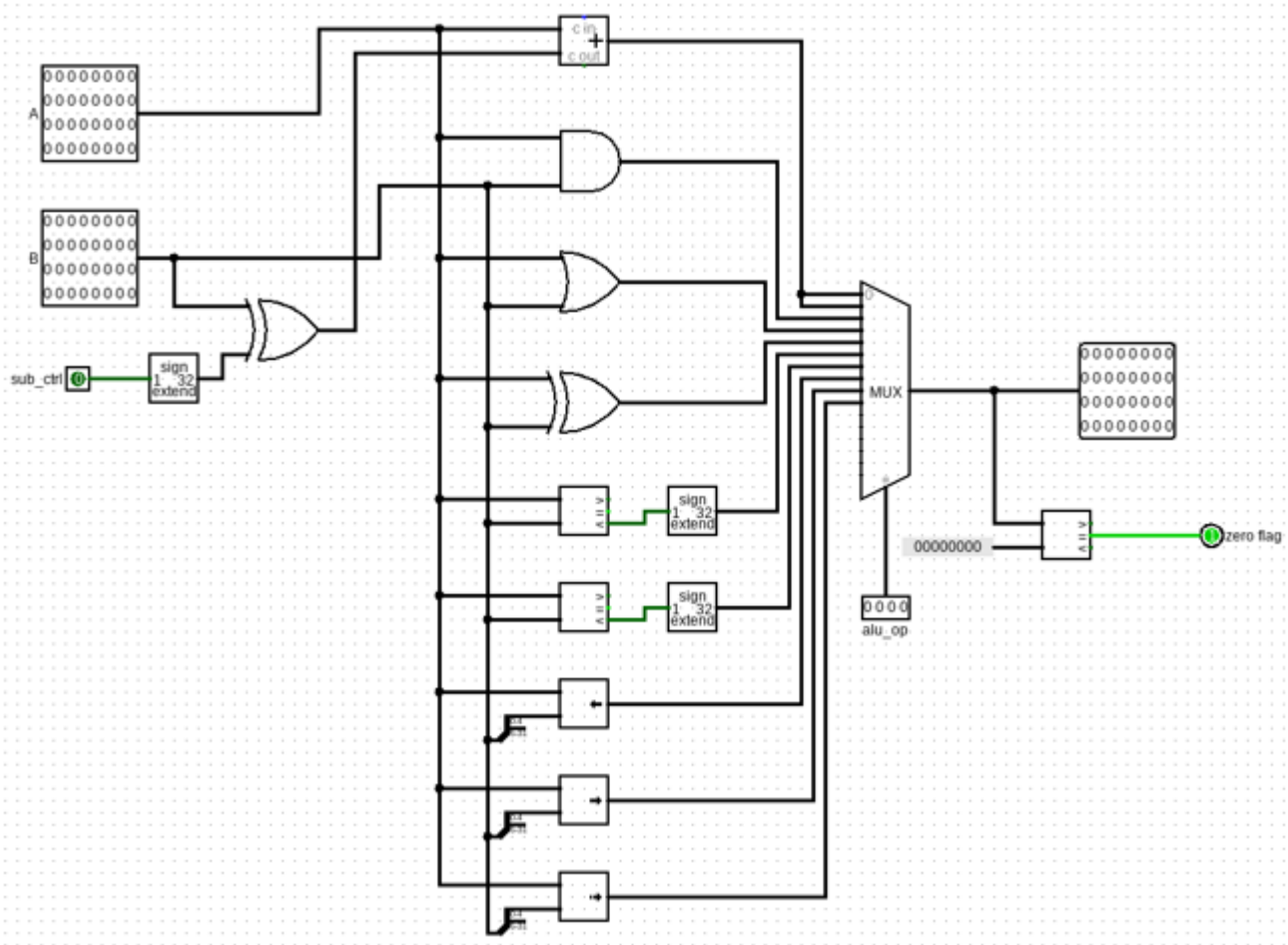
    initial begin
             A = 32'd15; B = 32'd10; alu_op = 4'b0000; // 15 + 10 = 25
        #10; A = 32'd15; B = 32'd10; alu_op = 4'b0001; // 15 - 10 = 5
        #10; A = 32'hF0F0; B = 32'hFF00; alu_op = 4'b0010; // F0F0 & FF00 = F000
        #10; A = 32'hF0F0; B = 32'h0F0F; alu_op = 4'b0011; // F0F0 | 0F0F = FFFF
        #10; A = 32'hFFFF; B = 32'h0FF0; alu_op = 4'b0100; // FFFF ^ 0FF0 = F00F
        #10; A = -32'd5; B = 32'd3; alu_op = 4'b0101; // -5 < 3 -> 1
        #10; A = 32'd4; B = 32'd2; alu_op = 4'b0111; // 4 << 2 = 16
        #10; A = 32'd10; B = 32'd10; alu_op = 4'b0001; // 10 - 10 = 0 -> zero=1
    end
```

```
endmodule
```

**Output:**



## Task 5:


A
B
sub_ctrl
sign 1 32 extend
c in
c out
MUX
sign 1 32 extend
sign 1 32 extend
00000000
alu_op
zero flag